

Import Required Libraries

```
from sklearn.metrics import roc_auc_score
import torch
import torch_geometric
import torch_sparse
import torch_scatter

print("Torch:", torch.__version__)
print("CUDA:", torch.cuda.is_available())
print("PyG extensions working")

Torch: 2.7.1+cu118
CUDA: True
PyG extensions working

import torch
print(torch.__version__)

2.7.1+cu118

import torch

print("Torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("GPU:", torch.cuda.get_device_name(0))

Torch: 2.7.1+cu118
CUDA available: True
GPU: NVIDIA GeForce RTX 3050 Laptop GPU

import torch
import torch.nn.functional as F

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

Using device: cuda

import torch
import torch_geometric

print(torch.__version__)
print("PyG working correctly")

2.7.1+cu118
PyG working correctly

import torch
import torch.nn.functional as F
from torch_geometric.nn import HeteroConv, SAGEConv
from sklearn.metrics import roc_auc_score
```

```

import torch

data = torch.load("data/fraud_graph.pt", weights_only=False)

print("Loaded graph:", data)

Loaded graph: HeteroData(
  account={
    x=[179590, 7],
    num_nodes=179590,
  },
  device={
    x=[99473, 1],
    num_nodes=99473,
  },
  ip={
    x=[99999, 1],
    num_nodes=99999,
  },
  location={ num_nodes=8 },
  merchant={ num_nodes=8 },
  (account, transaction, account)={
    edge_index=[2, 100000],
    edge_label=[100000],
    edge_attr=[100000, 1],
  },
  (account, uses_device, device)={ edge_index=[2, 100000] },
  (account, uses_ip, ip)={ edge_index=[2, 100000] },
  (account, located_at, location)={ edge_index=[2, 100000] },
  (account, purchases, merchant)={ edge_index=[2, 100000] }
)

edge_index = data["account", "transaction", "account"].edge_index
edge_label = data["account", "transaction", "account"].edge_label

import numpy as np
from sklearn.model_selection import train_test_split

num_edges = edge_label.shape[0]
indices = np.arange(num_edges)

train_idx, test_idx = train_test_split(
    indices,
    test_size=0.15,
    stratify=edge_label,
    random_state=42
)

train_idx, val_idx = train_test_split(
    train_idx,
    test_size=0.15,

```

```

        stratify=edge_label[train_idx],
        random_state=42
    )

import torch

data["account","transaction","account"].train_idx =
torch.tensor(train_idx)
data["account","transaction","account"].val_idx =
torch.tensor(val_idx)
data["account","transaction","account"].test_idx =
torch.tensor(test_idx)

```

Add Embeddings for Nodes Without Features

Our graph only has features for account nodes.

Other nodes need embeddings.

```

num_accounts = data["account"].num_nodes
num_devices = data["device"].num_nodes
num_ips = data["ip"].num_nodes
num_locations = data["location"].num_nodes
num_merchants = data["merchant"].num_nodes

```

Define the Heterogeneous GNN

```

class FraudGNN(torch.nn.Module):

    def __init__(self, hidden_channels):

        super().__init__()

        self.location_emb = torch.nn.Embedding(num_locations,
hidden_channels)
        self.merchant_emb = torch.nn.Embedding(num_merchants,
hidden_channels)

        self.conv1 = HeteroConv({
            ('account','transaction','account'): SAGEConv((-1,-1),
hidden_channels),
            ('account','uses_device','device'): SAGEConv((-1,-1),
hidden_channels),
            ('account','uses_ip','ip'): SAGEConv((-1,-1),
hidden_channels),
            ('account','located_at','location'): SAGEConv((-1,-1),
hidden_channels),
            ('account','purchases','merchant'): SAGEConv((-1,-1),
hidden_channels)
        })

```

```

        self.conv2 = HeteroConv({
            ('account', 'transaction', 'account'): SAGEConv((-1, -1),
hidden_channels),
            ('account', 'uses_device', 'device'): SAGEConv((-1, -1),
hidden_channels),
            ('account', 'uses_ip', 'ip'): SAGEConv((-1, -1),
hidden_channels),
            ('account', 'located_at', 'location'): SAGEConv((-1, -1),
hidden_channels),
            ('account', 'purchases', 'merchant'): SAGEConv((-1, -1),
hidden_channels)
        })

    def forward(self, data):
        x_dict = {
            "account": data["account"].x,
            "device": data["device"].x,
            "ip": data["ip"].x,
            "location": self.location_emb.weight,
            "merchant": self.merchant_emb.weight
        }

        edge_index_dict = data.edge_index_dict

        x_dict = self.conv1(x_dict, edge_index_dict)
        x_dict = {k: F.relu(v) for k, v in x_dict.items()}
        x_dict = {k: F.dropout(v, p=0.3, training=self.training) for
k, v in x_dict.items()}

        x_dict = self.conv2(x_dict, edge_index_dict)
        x_dict = {k: F.relu(v) for k, v in x_dict.items()}
        x_dict = {k: F.dropout(v, p=0.3, training=self.training) for
k, v in x_dict.items()}

        return x_dict

```

Edge Fraud Classifier

```

class EdgeClassifier(torch.nn.Module):

    def __init__(self, hidden_channels):
        super().__init__()

        self.lin1 = torch.nn.Linear(hidden_channels*2 + 1,
hidden_channels)
        self.lin2 = torch.nn.Linear(hidden_channels, 1)

```

```

def forward(self, z_src, z_dst, edge_attr):
    h = torch.cat([z_src, z_dst, edge_attr], dim=1)
    h = F.relu(self.lin1(h))
    return self.lin2(h).view(-1)

```

Initialize Model

```

hidden = 64

gnn = FraudGNN(hidden).to(device)
classifier = EdgeClassifier(hidden).to(device)

optimizer = torch.optim.Adam(
    list(gnn.parameters()) + list(classifier.parameters()),
    lr=0.001, weight_decay=1e-4
)

```

Training Loop LinkNeighborLoader

```

from torch_geometric.loader import LinkNeighborLoader

edge_index = data["account", "transaction", "account"].edge_index
edge_label = data["account", "transaction", "account"].edge_label

train_loader = LinkNeighborLoader(
    data,
    num_neighbors=[10, 10],
    edge_label_index=(
        ("account", "transaction", "account"),
        edge_index[:,
data["account", "transaction", "account"].train_idx]
    ),
    edge_label=edge_label[data["account", "transaction", "account"].train_idx],
    batch_size=2048,
    shuffle=True
)

test_loader = LinkNeighborLoader(
    data,
    num_neighbors=[10, 10],
    edge_label_index=(
        ("account", "transaction", "account"),
        edge_index[:,
data["account", "transaction", "account"].test_idx]
    ),

```

```

edge_label=edge_label[data["account","transaction","account"].test_idx
],
    batch_size=2048,
    shuffle=False
)
for epoch in range(20):
    gnn.train()
    classifier.train()
    total_loss = 0
    all_labels =
batch["account","transaction","account"].edge_label.float()

    pos_weight = ((len(all_labels)-
all_labels.sum())/all_labels.sum()).to(device)
    for batch in train_loader:

        batch = batch.to(device)    # ADD THIS LINE

        optimizer.zero_grad()

        x_dict = gnn(batch)

        z = x_dict["account"]

        src =
batch["account","transaction","account"].edge_label_index[0]
        dst =
batch["account","transaction","account"].edge_label_index[1]

        edge_attr =
batch["account","transaction","account"].edge_attr[:src.shape[0]]

        pred = classifier(z[src], z[dst], edge_attr)

        labels =
batch["account","transaction","account"].edge_label.float()

        loss = F.binary_cross_entropy_with_logits(
            pred,
            labels,
            pos_weight=pos_weight
        )

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

```

```
print(f"Epoch {epoch} Loss {total_loss:.4f}")
torch.save({
    "gnn": gnn.state_dict(),
    "classifier": classifier.state_dict()
}, "fraud_gnn_model.pt")
print("Model Saved")
```

```
Epoch 0 Loss 32.3625
Model Saved
Epoch 1 Loss 4.0352
Model Saved
Epoch 2 Loss 1.4387
Model Saved
Epoch 3 Loss 1.0276
Model Saved
Epoch 4 Loss 0.9089
Model Saved
Epoch 5 Loss 0.9642
Model Saved
Epoch 6 Loss 0.8980
Model Saved
Epoch 7 Loss 0.7859
Model Saved
Epoch 8 Loss 0.8503
Model Saved
Epoch 9 Loss 0.7073
Model Saved
Epoch 10 Loss 0.7015
Model Saved
Epoch 11 Loss 0.6057
Model Saved
Epoch 12 Loss 0.6961
Model Saved
Epoch 13 Loss 0.9068
Model Saved
Epoch 14 Loss 0.9687
Model Saved
Epoch 15 Loss 0.6598
Model Saved
Epoch 16 Loss 0.6452
Model Saved
Epoch 17 Loss 0.6518
Model Saved
Epoch 18 Loss 0.6938
Model Saved
Epoch 19 Loss 0.6303
Model Saved
```

Evaluate Model

```
!pip install matplotlib

Collecting matplotlib
  Downloading matplotlib-3.10.8-cp310-cp310-win_amd64.whl.metadata (52
kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl.metadata (5.5
kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.61.1-cp310-cp310-win_amd64.whl.metadata (116
kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.4.9-cp310-cp310-win_amd64.whl.metadata (6.4
kB)
Requirement already satisfied: numpy>=1.23 in c:\users\asus\anaconda3\
envs\gnn\lib\site-packages (from matplotlib) (2.2.6)
Requirement already satisfied: packaging>=20.0 in c:\users\asus\
anaconda3\envs\gnn\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\asus\anaconda3\
envs\gnn\lib\site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in c:\users\asus\
anaconda3\envs\gnn\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\asus\
anaconda3\envs\gnn\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\asus\anaconda3\
envs\gnn\lib\site-packages (from python-dateutil>=2.7->matplotlib)
(1.17.0)
Downloading matplotlib-3.10.8-cp310-cp310-win_amd64.whl (8.1 MB)
----- 0.0/8.1 MB ? eta -:--:--
----- 0.0/8.1 MB ? eta -:--:--
----- 0.0/8.1 MB ? eta -:--:--
----- 0.0/8.1 MB ? eta -:--:--
- 0.3/8.1 MB ? eta -:--:--
- 0.3/8.1 MB ? eta -:--:--
- 0.3/8.1 MB ? eta -:--:--
-- 0.5/8.1 MB 508.0 kB/s eta
0:00:15
-- 0.5/8.1 MB 508.0 kB/s eta
0:00:15
-- 0.5/8.1 MB 508.0 kB/s eta
0:00:15
-- 0.8/8.1 MB 459.5 kB/s eta
0:00:16
-- 0.8/8.1 MB 459.5 kB/s eta
0:00:16
-- 0.8/8.1 MB 459.5 kB/s eta
```

```
0:00:16
----- 1.0/8.1 MB 409.2 kB/s eta
0:00:18
----- 1.0/8.1 MB 409.2 kB/s eta
0:00:18
----- 1.0/8.1 MB 409.2 kB/s eta
0:00:18
----- 1.0/8.1 MB 409.2 kB/s eta
0:00:18
----- 1.3/8.1 MB 379.1 kB/s eta
0:00:18
----- 1.3/8.1 MB 379.1 kB/s eta
0:00:18
----- 1.3/8.1 MB 379.1 kB/s eta
0:00:18
----- 1.3/8.1 MB 379.1 kB/s eta
0:00:18
----- 1.3/8.1 MB 379.1 kB/s eta
0:00:18
----- 1.6/8.1 MB 376.1 kB/s eta
0:00:18
----- 1.6/8.1 MB 376.1 kB/s eta
0:00:18
----- 1.8/8.1 MB 401.0 kB/s eta
0:00:16
----- 1.8/8.1 MB 401.0 kB/s eta
0:00:16
----- 1.8/8.1 MB 401.0 kB/s eta
0:00:16
----- 1.8/8.1 MB 401.0 kB/s eta
0:00:16
----- 2.1/8.1 MB 392.8 kB/s eta
0:00:16
----- 2.1/8.1 MB 392.8 kB/s eta
0:00:16
----- 2.1/8.1 MB 392.8 kB/s eta
0:00:16
----- 2.4/8.1 MB 387.9 kB/s eta
0:00:15
----- 2.4/8.1 MB 387.9 kB/s eta
0:00:15
----- 2.4/8.1 MB 387.9 kB/s eta
0:00:15
----- 2.4/8.1 MB 387.9 kB/s eta
0:00:15
----- 2.6/8.1 MB 386.1 kB/s eta
0:00:15
----- 2.6/8.1 MB 386.1 kB/s eta
0:00:15
----- 2.6/8.1 MB 386.1 kB/s eta
0:00:15
```

-----	2.9/8.1 MB 388.4 kB/s eta
0:00:14	
-----	2.9/8.1 MB 388.4 kB/s eta
0:00:14	
-----	2.9/8.1 MB 388.4 kB/s eta
0:00:14	
-----	2.9/8.1 MB 388.4 kB/s eta
0:00:14	
-----	3.1/8.1 MB 382.1 kB/s eta
0:00:14	
-----	3.1/8.1 MB 382.1 kB/s eta
0:00:14	
-----	3.1/8.1 MB 382.1 kB/s eta
0:00:14	
-----	3.1/8.1 MB 382.1 kB/s eta
0:00:14	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.4/8.1 MB 376.3 kB/s eta
0:00:13	
-----	3.7/8.1 MB 346.8 kB/s eta
0:00:13	
-----	3.7/8.1 MB 346.8 kB/s eta
0:00:13	
-----	3.7/8.1 MB 346.8 kB/s eta
0:00:13	
-----	3.7/8.1 MB 346.8 kB/s eta
0:00:13	
-----	3.9/8.1 MB 340.9 kB/s eta
0:00:13	
-----	3.9/8.1 MB 340.9 kB/s eta
0:00:13	
-----	3.9/8.1 MB 340.9 kB/s eta
0:00:13	
-----	3.9/8.1 MB 340.9 kB/s eta
0:00:13	
-----	4.2/8.1 MB 334.7 kB/s eta

```
0:00:12
----- 4.2/8.1 MB 334.7 kB/s eta
0:00:12
----- 4.2/8.1 MB 334.7 kB/s eta
0:00:12
----- 4.5/8.1 MB 335.9 kB/s eta
0:00:11
----- 4.5/8.1 MB 335.9 kB/s eta
0:00:11
----- 4.5/8.1 MB 335.9 kB/s eta
0:00:11
----- 4.5/8.1 MB 335.9 kB/s eta
0:00:11
----- 4.7/8.1 MB 337.9 kB/s eta
0:00:11
----- 4.7/8.1 MB 337.9 kB/s eta
0:00:11
----- 5.0/8.1 MB 345.9 kB/s eta
0:00:10
----- 5.0/8.1 MB 345.9 kB/s eta
0:00:10
----- 5.0/8.1 MB 345.9 kB/s eta
0:00:10
----- 5.2/8.1 MB 348.0 kB/s eta
0:00:09
----- 5.2/8.1 MB 348.0 kB/s eta
0:00:09
----- 5.2/8.1 MB 348.0 kB/s eta
0:00:09
----- 5.5/8.1 MB 348.4 kB/s eta
0:00:08
----- 5.5/8.1 MB 348.4 kB/s eta
0:00:08
----- 5.5/8.1 MB 348.4 kB/s eta
0:00:08
----- 5.5/8.1 MB 348.4 kB/s eta
0:00:08
----- 5.5/8.1 MB 348.4 kB/s eta
0:00:08
----- 5.8/8.1 MB 346.4 kB/s eta
0:00:07
----- 5.8/8.1 MB 346.4 kB/s eta
0:00:07
----- 5.8/8.1 MB 346.4 kB/s eta
0:00:07
----- 5.8/8.1 MB 346.4 kB/s eta
0:00:07
----- 6.0/8.1 MB 344.9 kB/s eta
0:00:07
----- 6.0/8.1 MB 344.9 kB/s eta
0:00:07
```

```
----- 6.3/8.1 MB 351.1 kB/s eta
0:00:06
----- 6.3/8.1 MB 351.1 kB/s eta
0:00:06
----- 6.3/8.1 MB 351.1 kB/s eta
0:00:06
----- 6.3/8.1 MB 351.1 kB/s eta
0:00:06
----- 6.3/8.1 MB 351.1 kB/s eta
0:00:06
----- 6.6/8.1 MB 347.7 kB/s eta
0:00:05
----- 6.6/8.1 MB 347.7 kB/s eta
0:00:05
----- 6.8/8.1 MB 352.8 kB/s eta
0:00:04
----- 6.8/8.1 MB 352.8 kB/s eta
0:00:04
----- 7.1/8.1 MB 360.5 kB/s eta
0:00:03
----- 7.1/8.1 MB 360.5 kB/s eta
0:00:03
----- 7.3/8.1 MB 363.5 kB/s eta
0:00:03
----- 7.3/8.1 MB 363.5 kB/s eta
0:00:03
----- 7.6/8.1 MB 371.3 kB/s eta
0:00:02
----- 7.9/8.1 MB 379.8 kB/s eta
0:00:01
----- 8.1/8.1 MB 388.1 kB/s eta
0:00:01
----- 8.1/8.1 MB 386.1 kB/s
0:00:21
Downloading contourpy-1.3.2-cp310-cp310-win_amd64.whl (221 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.61.1-cp310-cp310-win_amd64.whl (1.6 MB)
----- 0.0/1.6 MB ? eta -:--:--
----- 0.3/1.6 MB ? eta -:--:--
----- 0.5/1.6 MB 1.3 MB/s eta
0:00:01
----- 0.5/1.6 MB 1.3 MB/s eta
0:00:01
----- 0.8/1.6 MB 1.0 MB/s eta
0:00:01
----- 1.0/1.6 MB 1.0 MB/s eta
0:00:01
----- 1.0/1.6 MB 1.0 MB/s eta
0:00:01
```



```

        recall_score,
        f1_score,
        accuracy_score,
        confusion_matrix,
        classification_report,
        average_precision_score
    )
    preds = []
    labels_all = []

    gnn.eval()
    classifier.eval()

    for batch in test_loader:
        batch = batch.to(device)
        with torch.no_grad():

            x_dict = gnn(batch)

            z = x_dict["account"]

            src =
batch["account", "transaction", "account"].edge_label_index[0]
            dst =
batch["account", "transaction", "account"].edge_label_index[1]

            edge_attr =
batch["account", "transaction", "account"].edge_attr[:src.shape[0]]

            pred = classifier(z[src], z[dst], edge_attr).sigmoid()

            preds.append(pred.cpu())

    labels_all.append(batch["account", "transaction", "account"].edge_label.
cpu())

    preds = torch.cat(preds).numpy()
    labels_all = torch.cat(labels_all).numpy()

    roc = roc_auc_score(labels_all, preds)
    pred_binary = (preds > 0.5).astype(int)
    print("Hetero GNN ROC-AUC:", roc)
    accuracy = accuracy_score(labels_all, pred_binary)
    precision = precision_score(labels_all, pred_binary)
    recall = recall_score(labels_all, pred_binary)
    f1 = f1_score(labels_all, pred_binary)

    print("\nClassification Metrics")
    print("-----")
    print("Accuracy :", accuracy)

```

```

print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)

cm = confusion_matrix(labels_all, pred_binary)

print("\nConfusion Matrix")
print("-----")
print(cm)

print("\nDetailed Classification Report")
print("-----")
print(classification_report(labels_all, pred_binary))

pr_auc = average_precision_score(labels_all, preds)

print("\nPrecision-Recall AUC:", pr_auc)

import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve

fpr, tpr, _ = roc_curve(labels_all, preds)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc:.4f}")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Fraud Detection")
plt.legend()
plt.show()

```

Hetero GNN ROC-AUC: 0.9995863956452317

Classification Metrics

```

-----
Accuracy : 0.9964666666666666
Precision: 0.834375
Recall   : 1.0
F1 Score : 0.909710391822828

```

Confusion Matrix

```

-----
[[14680   53]
 [    0  267]]

```

Detailed Classification Report

```

-----
              precision    recall  f1-score   support

0               1.00        1.00        1.00      14733

```

	1	0.83	1.00	0.91	267
accuracy				1.00	15000
macro avg		0.92	1.00	0.95	15000
weighted avg		1.00	1.00	1.00	15000
Precision-Recall AUC: 0.9784703887504251					

